

DSBDAL Assignment: Logistic Regression on Social Network Ads Dataset

Objective: Implement logistic regression to perform classification and compute confusion matrix metrics

Dataset: Social_Network_Ads.csv

Metrics to Calculate: TP, FP, TN, FN, Accuracy, Error Rate, Precision, Recall

Step 1: Import Libraries

```
# Import necessary libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix, accuracy_score, precision_score, recall_score

# Set display options
pd.set_option('display.max_columns', None)
sns.set_style('whitegrid')

print("Libraries imported successfully!")
```

Libraries imported successfully!

Step 2: Load the Dataset

```
# Load the Social_Network_Ads dataset
# Note: Update the file path according to your dataset location
df = pd.read_csv('Social_Network_Ads.csv')

print("Dataset loaded successfully!")
print(f"\nDataset Shape: {df.shape}")
print(f"\nFirst few rows:")
df.head()
```

Dataset loaded successfully!

Dataset Shape: (400, 5)

First few rows:

	User ID	Gender	Age	EstimatedSalary	Purchased
0	15624510	Male	19	19000	0
1	15810944	Male	35	20000	0
2	15668575	Female	26	43000	0
3	15603246	Female	27	57000	0
4	15804002	Male	19	76000	0

Next steps:

[Generate code with df](#)

[New interactive sheet](#)

Step 3: Exploratory Data Analysis (EDA)

```
# Display basic information about the dataset
print("Dataset Information:")
print(df.info())
print("\n" + "="*50)
print("\nStatistical Summary:")
print(df.describe())
print("\n" + "="*50)
print("\nColumn Names:")
print(df.columns.tolist())
```

Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 400 entries, 0 to 399

```
Data columns (total 5 columns):
#   Column                Non-Null Count  Dtype
---  -
0   User ID                400 non-null  int64
1   Gender                  400 non-null  object
2   Age                     400 non-null  int64
3   EstimatedSalary         400 non-null  int64
4   Purchased               400 non-null  int64
dtypes: int64(4), object(1)
memory usage: 15.8+ KB
None

=====

Statistical Summary:
      User ID      Age  EstimatedSalary  Purchased
count  4.000000e+02  400.000000      400.000000  400.000000
mean   1.569154e+07  37.655000     69742.500000  0.357500
std    7.165832e+04  10.482877     34096.960282  0.479864
min    1.556669e+07  18.000000     15000.000000  0.000000
25%    1.562676e+07  29.750000     43000.000000  0.000000
50%    1.569434e+07  37.000000     70000.000000  0.000000
75%    1.575036e+07  46.000000     88000.000000  1.000000
max    1.581524e+07  60.000000    150000.000000  1.000000

=====

Column Names:
['User ID', 'Gender', 'Age', 'EstimatedSalary', 'Purchased']
```

```
# Check for missing values
print("Missing Values:")
print(df.isnull().sum())
print("\nTotal Missing Values:", df.isnull().sum().sum())
```

Missing Values:

```
User ID      0
Gender       0
Age          0
EstimatedSalary  0
Purchased    0
dtype: int64
```

Total Missing Values: 0

```
# Check target variable distribution
print("Target Variable Distribution:")
if 'Purchased' in df.columns:
    print(df['Purchased'].value_counts())
    print("\nPercentage Distribution:")
    print(df['Purchased'].value_counts(normalize=True) * 100)
```

Target Variable Distribution:

```
Purchased
0    257
1    143
Name: count, dtype: int64
```

Percentage Distribution:

```
Purchased
0    64.25
1    35.75
Name: proportion, dtype: float64
```

▼ Step 4: Data Visualization

```
# Visualize the data
fig, axes = plt.subplots(2, 2, figsize=(14, 10))

# Assuming the dataset has Age, EstimatedSalary, and Purchased columns
# Adjust column names if different

# Plot 1: Age distribution
if 'Age' in df.columns:
    axes[0, 0].hist(df['Age'], bins=20, color='skyblue', edgecolor='black')
    axes[0, 0].set_title('Age Distribution')
    axes[0, 0].set_xlabel('Age')
    axes[0, 0].set_ylabel('Frequency')

# Plot 2: Estimated Salary distribution
if 'EstimatedSalary' in df.columns:
    axes[0, 1].hist(df['EstimatedSalary'], bins=20, color='lightgreen', edgecolor='black')
    axes[0, 1].set_title('Estimated Salary Distribution')
```

```

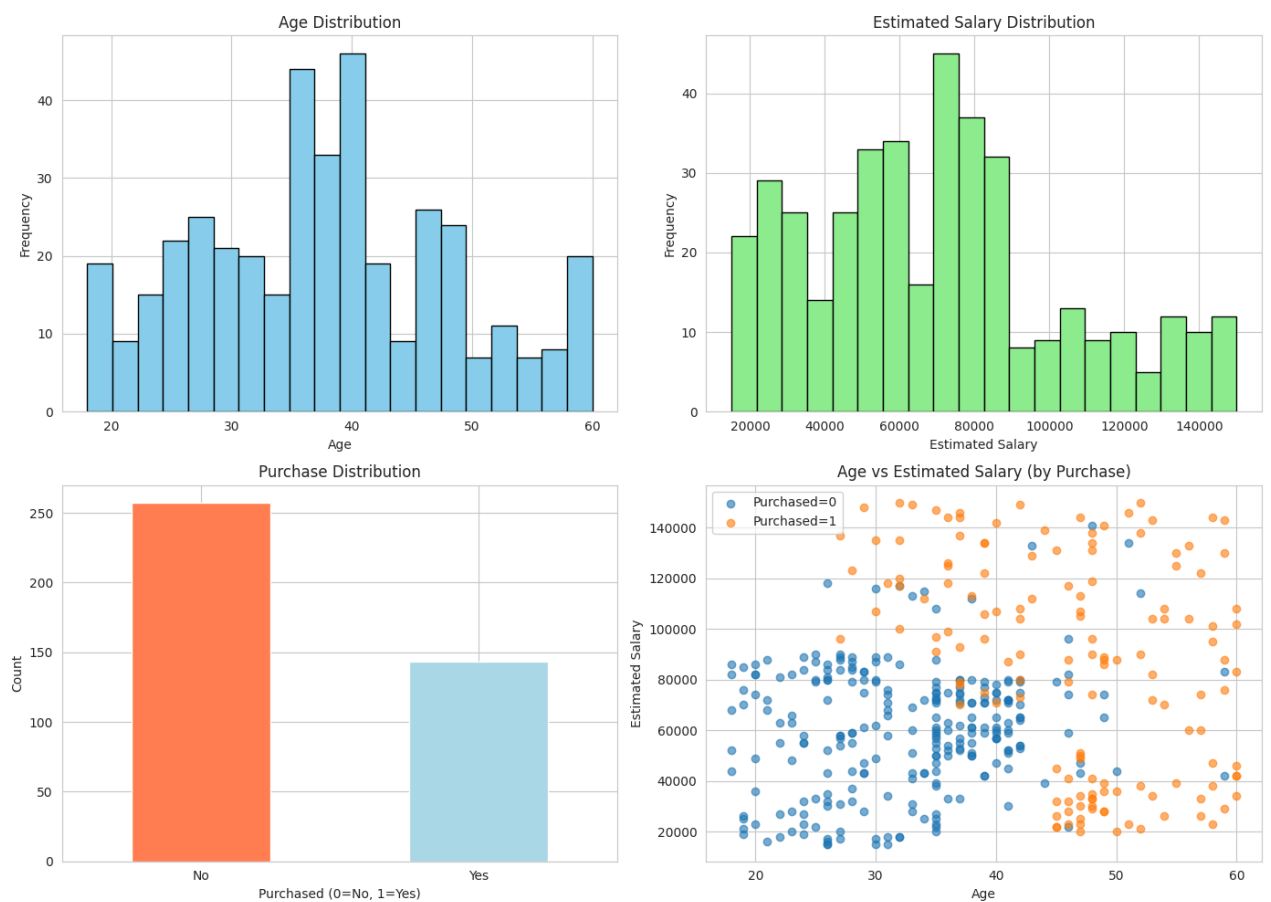
axes[0, 1].set_xlabel('Estimated Salary')
axes[0, 1].set_ylabel('Frequency')

# Plot 3: Purchase distribution
if 'Purchased' in df.columns:
    df['Purchased'].value_counts().plot(kind='bar', ax=axes[1, 0], color=['coral', 'lightblue'])
    axes[1, 0].set_title('Purchase Distribution')
    axes[1, 0].set_xlabel('Purchased (0=No, 1=Yes)')
    axes[1, 0].set_ylabel('Count')
    axes[1, 0].set_xticklabels(['No', 'Yes'], rotation=0)

# Plot 4: Age vs Salary colored by Purchase
if all(col in df.columns for col in ['Age', 'EstimatedSalary', 'Purchased']):
    for purchase in [0, 1]:
        subset = df[df['Purchased'] == purchase]
        axes[1, 1].scatter(subset['Age'], subset['EstimatedSalary'],
                           label=f'Purchased={purchase}', alpha=0.6)
    axes[1, 1].set_title('Age vs Estimated Salary (by Purchase)')
    axes[1, 1].set_xlabel('Age')
    axes[1, 1].set_ylabel('Estimated Salary')
    axes[1, 1].legend()

plt.tight_layout()
plt.show()

```



Step 5: Data Preprocessing

```

# Select features and target variable
# Typical Social_Network_Ads has: User ID, Gender, Age, EstimatedSalary, Purchased
# We'll use Age and EstimatedSalary as features

# Drop User ID if present (not useful for prediction)
if 'User ID' in df.columns:
    df = df.drop('User ID', axis=1)

```

```
# Convert categorical variables if present (e.g., Gender)
if 'Gender' in df.columns:
    df['Gender'] = df['Gender'].map({'Male': 1, 'Female': 0})
    print("Gender converted to numerical values")

print("\nPreprocessed Dataset:")
df.head()
```

Gender converted to numerical values

Preprocessed Dataset:

	Gender	Age	EstimatedSalary	Purchased
0	1	19	19000	0
1	1	35	20000	0
2	0	26	43000	0
3	0	27	57000	0
4	1	19	76000	0

Next steps:

[Generate code with df](#)

[New interactive sheet](#)

```
# Check correlation matrix
print("Correlation Matrix:")
correlation_matrix = df.corr()
print(correlation_matrix)

# Visualize correlation matrix
plt.figure(figsize=(8, 6))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', center=0,
            square=True, linewidths=1)
plt.title('Correlation Matrix')
plt.show()
```

Correlation Matrix:

```

                Gender      Age  EstimatedSalary  Purchased
Gender    1.000000  -0.073741  -0.060435  -0.042469
Age       -0.073741  1.000000   0.155238   0.622454
EstimatedSalary -0.060435  0.155238   1.000000   0.362083
Purchased  -0.042469  0.622454   0.362083   1.000000

```



```
# Separate features (X) and target variable (y)
# Adjust column names based on your actual dataset
X = df.drop('Purchased', axis=1) # Features
y = df['Purchased']              # Target

print(f"Features shape: {X.shape}")
print(f"Target shape: {y.shape}")
print(f"\nFeature columns: {X.columns.tolist()}")
```

Features shape: (400, 3)
Target shape: (400,)

```
Feature columns: ['Gender', 'Age', 'EstimatedSalary']
```

▼ Step 6: Split the Dataset

```
# Split the dataset into training and testing sets (80-20 split)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

print(f"Training set size: {X_train.shape[0]} samples")
print(f"Testing set size: {X_test.shape[0]} samples")
print(f"\nTraining set shape: {X_train.shape}")
print(f"Testing set shape: {X_test.shape}")
```

```
Training set size: 320 samples
Testing set size: 80 samples
```

```
Training set shape: (320, 3)
Testing set shape: (80, 3)
```

▼ Step 7: Feature Scaling

```
# Apply feature scaling using StandardScaler
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print("Feature scaling completed!")
print(f"\nScaled Training Data (first 5 rows):")
print(X_train_scaled[:5])
```

```
Feature scaling completed!
```

```
Scaled Training Data (first 5 rows):
[[-1.          -1.06675246 -0.38634438]
 [ 1.           0.79753468 -1.22993871]
 [-1.           0.11069205  1.853544   ]
 [-1.           0.60129393 -0.90995465]
 [-1.           1.87685881 -1.28811763]]
```

▼ Step 8: Build and Train Logistic Regression Model

```
# Create and train the Logistic Regression model
logreg = LogisticRegression(random_state=42, max_iter=1000)

# Train the model
logreg.fit(X_train_scaled, y_train)

print("Logistic Regression Model trained successfully!")
print(f"\nModel Coefficients: {logreg.coef_}")
print(f"Model Intercept: {logreg.intercept_}")
```

```
Logistic Regression Model trained successfully!
```

```
Model Coefficients: [[0.13721002 1.98963144 1.14512193]]
Model Intercept: [-1.00504843]
```

▼ Step 9: Make Predictions

```
# Make predictions on training set
y_train_pred = logreg.predict(X_train_scaled)

# Make predictions on testing set
y_test_pred = logreg.predict(X_test_scaled)

print("Predictions completed!")
print(f"\nFirst 10 Test Predictions: {y_test_pred[:10]}")
print(f"First 10 Actual Values: {y_test.values[:10]}")
```

```
Predictions completed!
```

```
First 10 Test Predictions: [0 1 0 1 0 0 1 0 0 0]
First 10 Actual Values: [0 1 0 1 0 0 1 0 0 0]
```

```
# Get prediction probabilities
y_test_prob = logreg.predict_proba(X_test_scaled)
```

```
print("Prediction Probabilities (first 5 samples):")
print("Class 0 (No Purchase) | Class 1 (Purchase)")
for i in range(5):
    print(f"{y_test_prob[i][0]:.4f}          | {y_test_prob[i][1]:.4f}")
```

```
Prediction Probabilities (first 5 samples):
Class 0 (No Purchase) | Class 1 (Purchase)
0.7620                | 0.2380
0.0273                | 0.9727
0.9810                | 0.0190
0.1556                | 0.8444
0.9864                | 0.0136
```

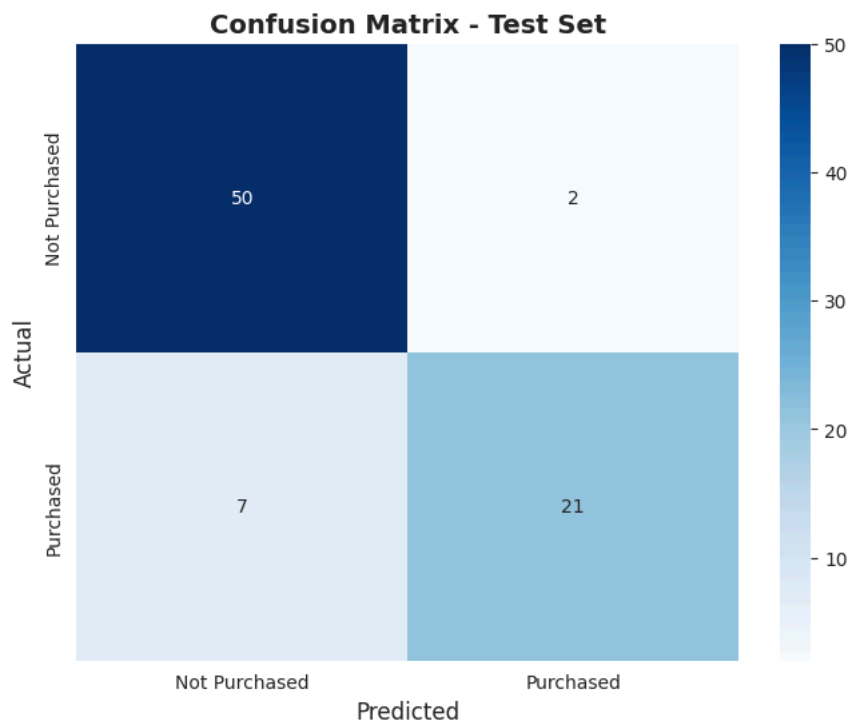
Step 10: Confusion Matrix

```
# Compute confusion matrix for test set
cm_test = confusion_matrix(y_test, y_test_pred)

print("Confusion Matrix (Test Set):")
print(cm_test)
print("\n")

# Visualize confusion matrix
plt.figure(figsize=(8, 6))
sns.heatmap(cm_test, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Not Purchased', 'Purchased'],
            yticklabels=['Not Purchased', 'Purchased'])
plt.title('Confusion Matrix - Test Set', fontsize=14, fontweight='bold')
plt.ylabel('Actual', fontsize=12)
plt.xlabel('Predicted', fontsize=12)
plt.show()
```

```
Confusion Matrix (Test Set):
[[50  2]
 [ 7 21]]
```



Step 11: Extract Confusion Matrix Components

```
# Extract TP, TN, FP, FN from confusion matrix
TN = cm_test[0, 0] # True Negative
FP = cm_test[0, 1] # False Positive
FN = cm_test[1, 0] # False Negative
TP = cm_test[1, 1] # True Positive

print("="*50)
print("CONFUSION MATRIX COMPONENTS")
```

```

print("="*50)
print(f"True Positive (TP): {TP}")
print(f"True Negative (TN): {TN}")
print(f"False Positive (FP): {FP}")
print(f"False Negative (FN): {FN}")
print("="*50)

```

```

=====
CONFUSION MATRIX COMPONENTS
=====
True Positive (TP):  21
True Negative (TN):  50
False Positive (FP):  2
False Negative (FN):  7
=====

```

Step 12: Calculate Evaluation Metrics

```

# Calculate metrics manually
# Accuracy = (TP + TN) / (TP + TN + FP + FN)
accuracy_manual = (TP + TN) / (TP + TN + FP + FN)

# Error Rate = (FP + FN) / (TP + TN + FP + FN)
error_rate_manual = (FP + FN) / (TP + TN + FP + FN)

# Precision = TP / (TP + FP)
precision_manual = TP / (TP + FP) if (TP + FP) > 0 else 0

# Recall (Sensitivity) = TP / (TP + FN)
recall_manual = TP / (TP + FN) if (TP + FN) > 0 else 0

# F1-Score = 2 * (Precision * Recall) / (Precision + Recall)
f1_score_manual = 2 * (precision_manual * recall_manual) / (precision_manual + recall_manual) if (precision_manual + recall_manual) > 0 else 0

print("="*50)
print("EVALUATION METRICS (Manual Calculation)")
print("="*50)
print(f"Accuracy:      {accuracy_manual:.4f} ({accuracy_manual*100:.2f}%)")
print(f"Error Rate:     {error_rate_manual:.4f} ({error_rate_manual*100:.2f}%)")
print(f"Precision:      {precision_manual:.4f} ({precision_manual*100:.2f}%)")
print(f"Recall:         {recall_manual:.4f} ({recall_manual*100:.2f}%)")
print(f"F1-Score:       {f1_score_manual:.4f} ({f1_score_manual*100:.2f}%)")
print("="*50)

```

```

=====
EVALUATION METRICS (Manual Calculation)
=====
Accuracy:      0.8875 (88.75%)
Error Rate:    0.1125 (11.25%)
Precision:     0.9130 (91.30%)
Recall:        0.7500 (75.00%)
F1-Score:      0.8235 (82.35%)
=====

```

```

# Verify with sklearn functions
accuracy_sklearn = accuracy_score(y_test, y_test_pred)
precision_sklearn = precision_score(y_test, y_test_pred)
recall_sklearn = recall_score(y_test, y_test_pred)

print("="*50)
print("VERIFICATION WITH SKLEARN")
print("="*50)
print(f"Accuracy (sklearn): {accuracy_sklearn:.4f} ({accuracy_sklearn*100:.2f}%)")
print(f"Precision (sklearn): {precision_sklearn:.4f} ({precision_sklearn*100:.2f}%)")
print(f"Recall (sklearn):    {recall_sklearn:.4f} ({recall_sklearn*100:.2f}%)")
print("="*50)

```

```

=====
VERIFICATION WITH SKLEARN
=====
Accuracy (sklearn):  0.8875 (88.75%)
Precision (sklearn): 0.9130 (91.30%)
Recall (sklearn):    0.7500 (75.00%)
=====

```

Step 13: Training Set Performance

```

# Evaluate on training set
cm_train = confusion_matrix(y_train, y_train_pred)

```

```

# Extract components
TN_train = cm_train[0, 0]
FP_train = cm_train[0, 1]
FN_train = cm_train[1, 0]
TP_train = cm_train[1, 1]

# Calculate metrics
accuracy_train = (TP_train + TN_train) / (TP_train + TN_train + FP_train + FN_train)
precision_train = TP_train / (TP_train + FP_train) if (TP_train + FP_train) > 0 else 0
recall_train = TP_train / (TP_train + FN_train) if (TP_train + FN_train) > 0 else 0

print("="*50)
print("TRAINING SET PERFORMANCE")
print("="*50)
print("\nConfusion Matrix:")
print(cm_train)
print(f"\nAccuracy: {accuracy_train:.4f} ({accuracy_train*100:.2f}%)")
print(f"Precision: {precision_train:.4f} ({precision_train*100:.2f}%)")
print(f"Recall: {recall_train:.4f} ({recall_train*100:.2f}%)")
print("="*50)

```

```

=====
TRAINING SET PERFORMANCE
=====

Confusion Matrix:
[[189  16]
 [ 43  72]]

Accuracy:  0.8156 (81.56%)
Precision: 0.8182 (81.82%)
Recall:    0.6261 (62.61%)
=====

```

Step 14: Comprehensive Performance Summary

```

# Create a summary DataFrame
summary_data = {
    'Metric': ['True Positive (TP)', 'True Negative (TN)', 'False Positive (FP)',
              'False Negative (FN)', 'Accuracy', 'Error Rate', 'Precision', 'Recall'],
    'Training Set': [TP_train, TN_train, FP_train, FN_train,
                    f"{accuracy_train:.4f}", f"{1-accuracy_train:.4f}",
                    f"{precision_train:.4f}", f"{recall_train:.4f}"],
    'Test Set': [TP, TN, FP, FN,
                f"{accuracy_manual:.4f}", f"{error_rate_manual:.4f}",
                f"{precision_manual:.4f}", f"{recall_manual:.4f}"]
}

summary_df = pd.DataFrame(summary_data)

print("\n" + "="*70)
print("COMPREHENSIVE PERFORMANCE SUMMARY")
print("="*70)
print(summary_df.to_string(index=False))
print("="*70)

```

```

=====
COMPREHENSIVE PERFORMANCE SUMMARY
=====

```

Metric	Training Set	Test Set
True Positive (TP)	72	21
True Negative (TN)	189	50
False Positive (FP)	16	2
False Negative (FN)	43	7
Accuracy	0.8156	0.8875
Error Rate	0.1844	0.1125
Precision	0.8182	0.9130
Recall	0.6261	0.7500

```

=====

```

Step 15: Visualization of Results

```

# Plot comparison of metrics
metrics = ['Accuracy', 'Precision', 'Recall']
train_scores = [accuracy_train, precision_train, recall_train]
test_scores = [accuracy_manual, precision_manual, recall_manual]

x = np.arange(len(metrics))

```



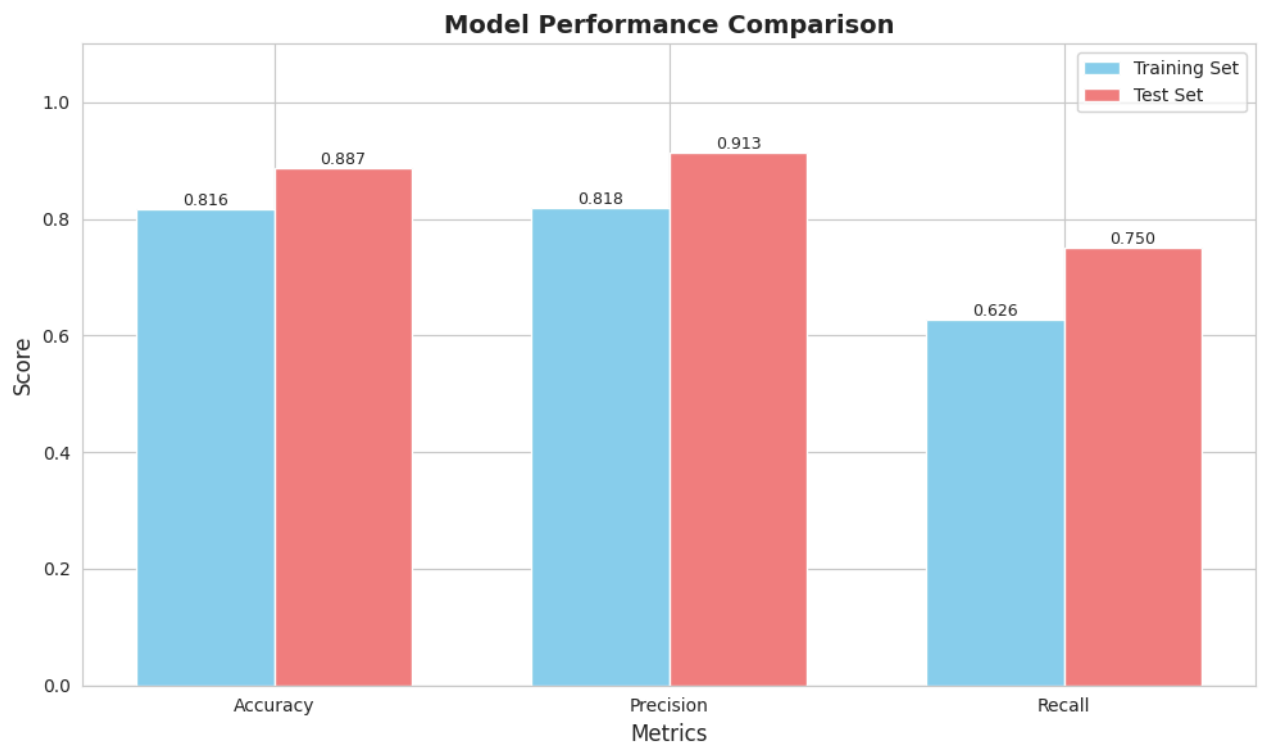
```
width = 0.35

fig, ax = plt.subplots(figsize=(10, 6))
bars1 = ax.bar(x - width/2, train_scores, width, label='Training Set', color='skyblue')
bars2 = ax.bar(x + width/2, test_scores, width, label='Test Set', color='lightcoral')

ax.set_xlabel('Metrics', fontsize=12)
ax.set_ylabel('Score', fontsize=12)
ax.set_title('Model Performance Comparison', fontsize=14, fontweight='bold')
ax.set_xticks(x)
ax.set_xticklabels(metrics)
ax.legend()
ax.set_ylim([0, 1.1])

# Add value labels on bars
for bars in [bars1, bars2]:
    for bar in bars:
        height = bar.get_height()
        ax.text(bar.get_x() + bar.get_width()/2., height,
                f'{height:.3f}', ha='center', va='bottom', fontsize=9)

plt.tight_layout()
plt.show()
```



Step 16: Classification Report

```
from sklearn.metrics import classification_report

print("="*50)
print("DETAILED CLASSIFICATION REPORT - TEST SET")
print("="*50)
print(classification_report(y_test, y_test_pred,
                           target_names=['Not Purchased', 'Purchased']))
print("="*50)
```

```
=====
DETAILED CLASSIFICATION REPORT - TEST SET
=====
              precision    recall  f1-score   support

Not Purchased      0.88      0.96      0.92         52
Purchased          0.91      0.75      0.82         28

   accuracy      0.89
  macro avg      0.90
weighted avg      0.89
```

✓ Conclusion

In this assignment, we have:

1. Successfully loaded and explored the Social_Network_Ads dataset
2. Performed data preprocessing including handling categorical variables and feature scaling
3. Implemented Logistic Regression for binary classification
4. Computed the Confusion Matrix and extracted TP, TN, FP, FN values
5. Calculated important evaluation metrics:
 - **Accuracy:** Overall correctness of the model
 - **Error Rate:** Overall incorrectness of the model
 - **Precision:** Accuracy of positive predictions
 - **Recall:** Coverage of actual positive cases
6. Evaluated model performance on both training and testing sets
7. Visualized the results for better understanding

The Logistic Regression model has been successfully trained and evaluated on the Social Network Ads dataset!

Start coding or generate with AI.